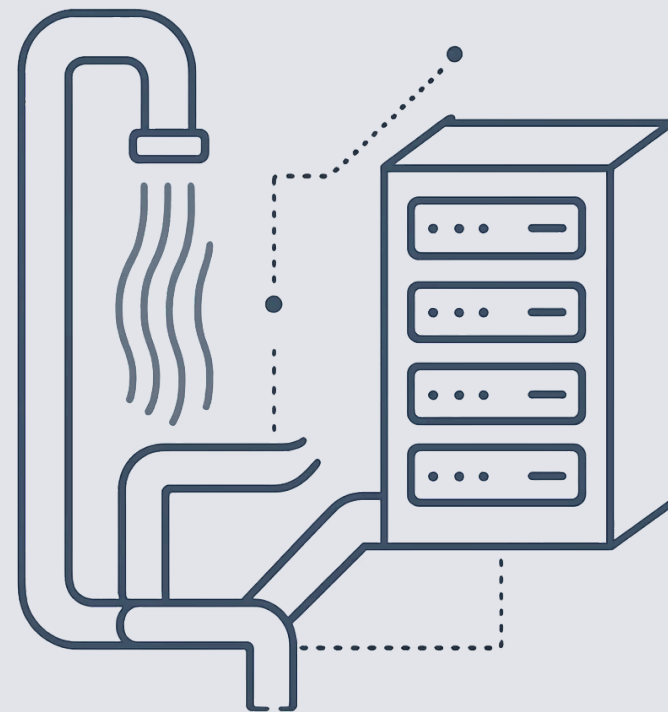


Programmation Réactive avec Spring WebFlux



Au programme

Ce que vous allez apprendre

01

Principes réactifs

Comprendre les bases du paradigme réactif

02

Project Reactor

Découvrir la bibliothèque au cœur de WebFlux

03

WebFlux vs MVC

Comparer les deux approches Spring

04

Flux, Mono & Opérateurs

Maîtriser les blocs fondamentaux

05

Travaux pratiques

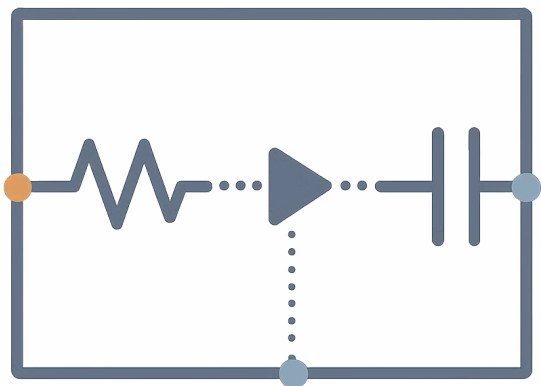
Implémenter votre première application WebFlux

Partie 1

Principes de la programmation réactive



Qu'est-ce que la programmation réactive ?



La programmation réactive est un paradigme orienté autour des **flux de données asynchrones** et de la **propagation du changement**.

Au lieu d'attendre qu'une opération se termine (style impératif), on **réagit** aux événements au fur et à mesure qu'ils arrivent.

- i Pensez à un tableau Excel : quand une cellule change, toutes les cellules qui en dépendent se mettent à jour automatiquement.

Le problème que ça résout

Approche traditionnelle (bloquante)

Le thread attend la réponse de la base de données, du réseau, du fichier... Il est **bloqué** et inutilisable pendant ce temps.

⚠ Avec 1 000 requêtes simultanées, il faut 1 000 threads — une consommation mémoire énorme.

Approche réactive (non-bloquante)

Le thread lance une opération et **continue à travailler**. Quand le résultat arrive, il est notifié et traite la réponse.

✅ Quelques threads suffisent pour gérer des milliers de requêtes simultanées.

Le Manifeste Réactif

Quatre principes fondamentaux définissent un système réactif moderne.



Réactif

Temps de réponse rapide et prévisible



Résilient

Reste fonctionnel malgré les pannes



Élastique

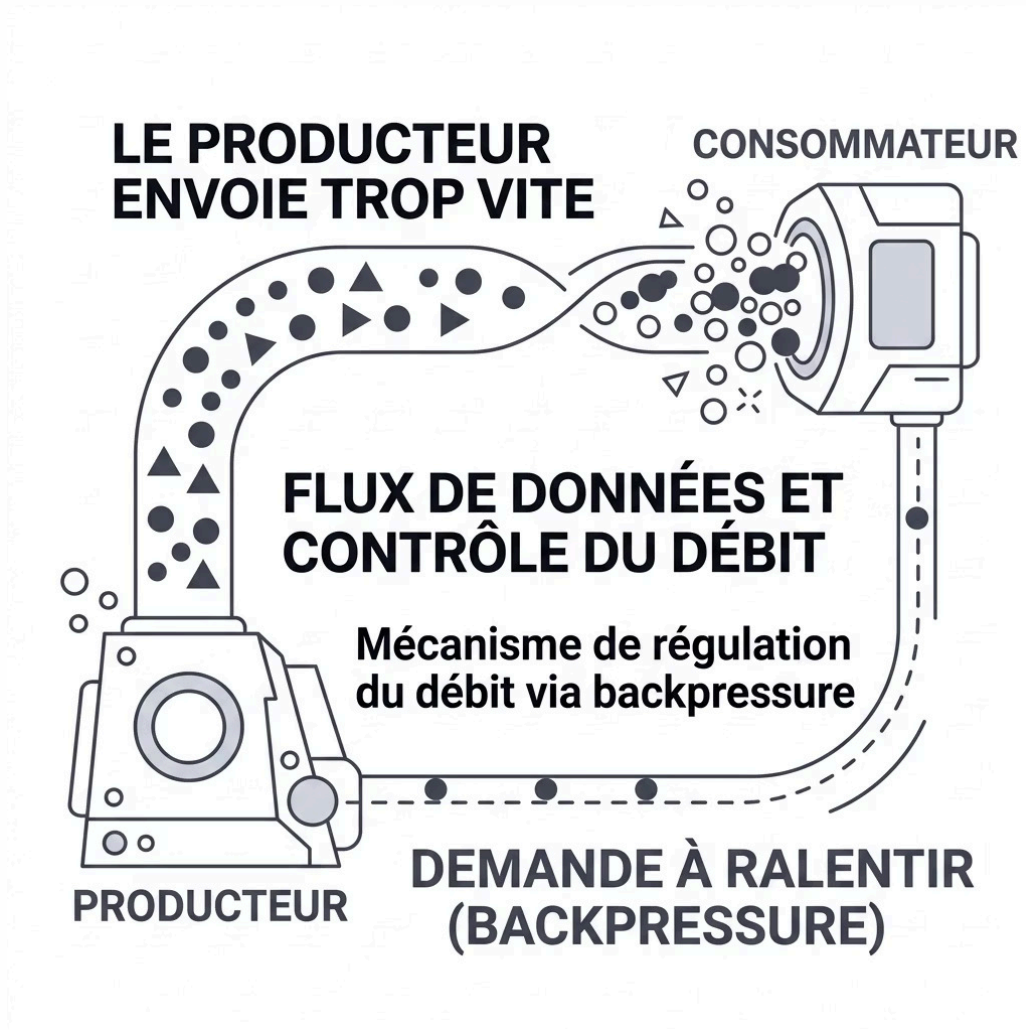
S'adapte à la charge dynamiquement



Orienté messages

Communication asynchrone par messages

La notion de Backpressure



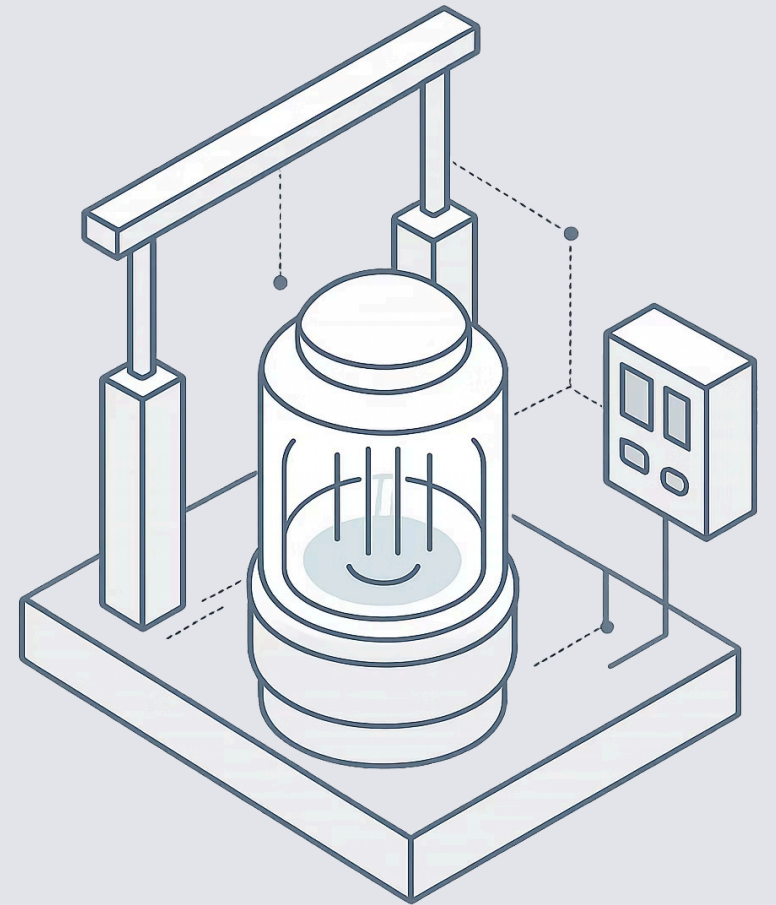
Le **backpressure** est un mécanisme clé de la programmation réactive. Il permet au **consommateur** de signaler au **producteur** qu'il ne peut pas traiter les données plus vite.

Sans backpressure, un producteur trop rapide peut saturer la mémoire et faire planter l'application.

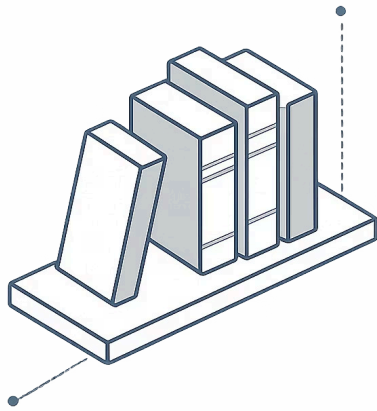
i C'est comme ouvrir le robinet juste assez fort pour ne pas déborder le verre.

Partie 2

Project Reactor



Project Reactor en bref

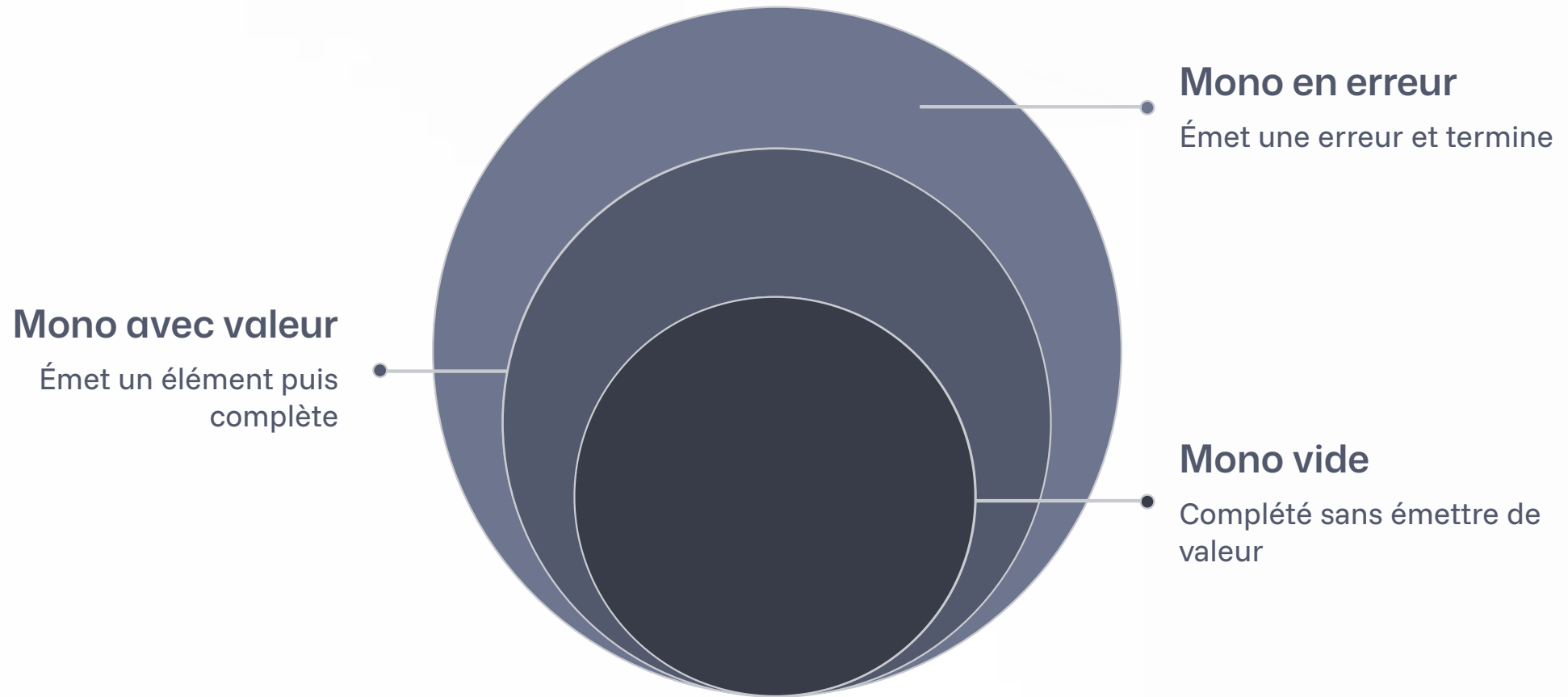


Project Reactor est la bibliothèque réactive développée par Pivotal (VMware), qui sert de fondation à Spring WebFlux.

- Basé sur la spécification **Reactive Streams**
- Deux types principaux : **Mono** et **Flux**
- Riche bibliothèque d'**opérateurs** fonctionnels
- Intégration native avec l'écosystème **Spring**

Mono : 0 ou 1 élément

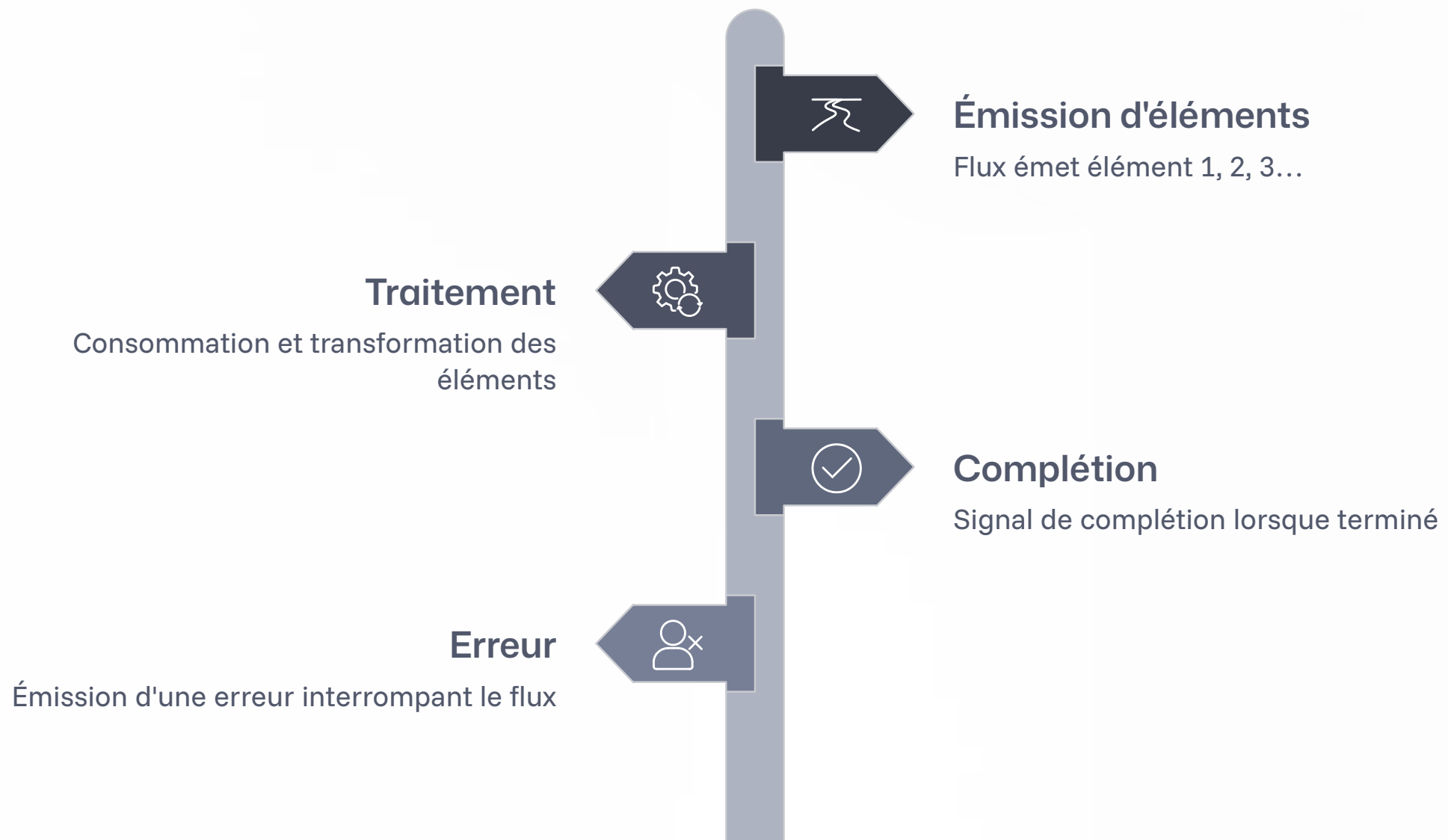
Un **Mono** représente un flux asynchrone qui émet **au plus un élément**, puis se termine (avec succès ou avec une erreur).



Utilisez `Mono` pour les opérations qui retournent un seul résultat : recherche par ID, création d'une entité, appel HTTP...

Flux : 0 à N éléments

Un **Flux** représente un flux asynchrone qui peut émettre **plusieurs éléments** au fil du temps, puis se termine.



Utilisez `Flux` pour les listes, les streams d'événements en temps réel, les résultats de requêtes multiples...

Mono vs Flux – Le résumé visuel

Mono<T>

0 ou 1 élément

- Recherche par ID
- Sauvegarde en base
- Appel HTTP unique

Flux<T>

0 à N éléments

- Liste d'utilisateurs
- Stream d'événements
- Résultats de recherche

📌 Règle simple : **un seul résultat attendu ?** → Mono. **Plusieurs résultats ?** → Flux.



Partie 3

WebFlux versus Spring MVC

Deux philosophies, un même but

Spring MVC

Modèle bloquant — Un thread par requête. Simple, mature, idéal pour les applications CRUD classiques avec une charge modérée.

- Serveur Tomcat (par défaut)
- API synchrone et intuitive
- Écosystème très riche

Spring WebFlux

Modèle non-bloquant — Peu de threads, beaucoup de requêtes. Idéal pour les applications à haute concurrence et les architectures microservices.

- Serveur Netty (par défaut)
- API réactive (Mono/Flux)
- Performances à grande échelle

Quand choisir WebFlux ?



Haute concurrence

Des milliers de requêtes simultanées à gérer avec un minimum de ressources serveur.



Streaming temps réel

Notifications push, flux d'événements SSE, WebSockets... la data arrive en continu.



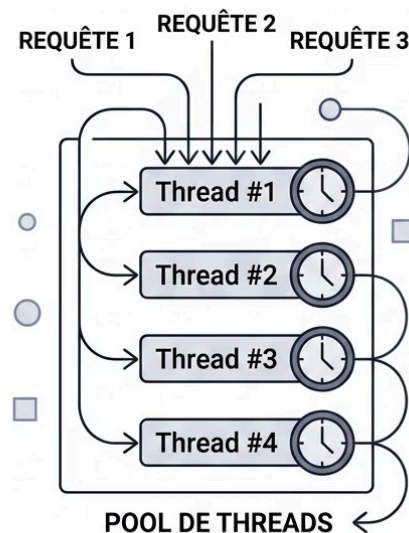
Microservices

Orchestration de multiples appels API en parallèle sans bloquer les threads.

 WebFlux n'est pas forcément meilleur que MVC — c'est un outil différent, adapté à des besoins spécifiques.

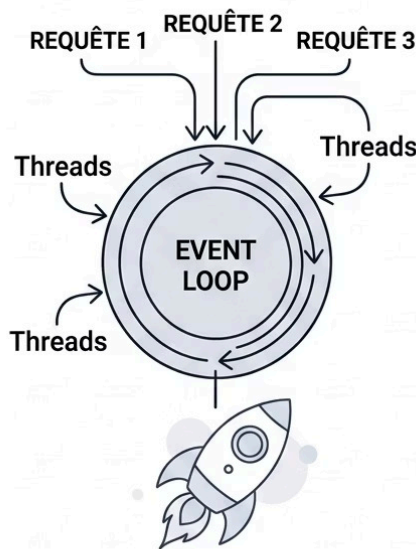
Architecture comparée

MVC & Thread-per-Request



Un thread par requête;
chaque requête bloque.

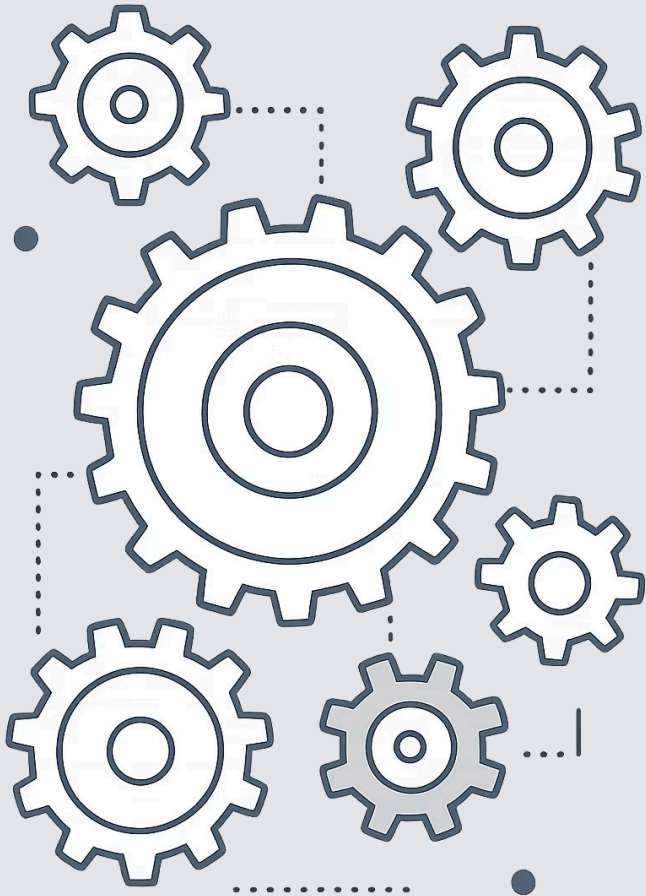
WebFlux & Event Loop



Quelques threads;
traitement asynchrone
concurant.

Dans Spring MVC, chaque requête **monopolise un thread** jusqu'à la fin du traitement. Le pool de threads est le facteur limitant.

Dans WebFlux, une **boucle d'événements** (Event Loop) distribue le travail. Les threads ne restent jamais inactifs à attendre.



Partie 4

Les Opérateurs Réactifs

Transformer les données avec les opérateurs

Les opérateurs permettent de **composer** des pipelines de traitement sur vos Flux et Mono, de façon déclarative et lisible.

1

map

Transforme chaque élément en un autre. `flux.map(s -> s.toUpperCase())`

2

filter

Garde uniquement les éléments qui satisfont une condition. `flux.filter(n -> n > 0)`

3

flatMap

Transforme et aplatit des flux imbriqués. Idéal pour les appels asynchrones enchaînés.

Autres opérateurs essentiels

zip

Combine deux Flux en associant leurs éléments deux à deux.

merge

Fusionne plusieurs Flux en un seul, dans l'ordre d'arrivée.

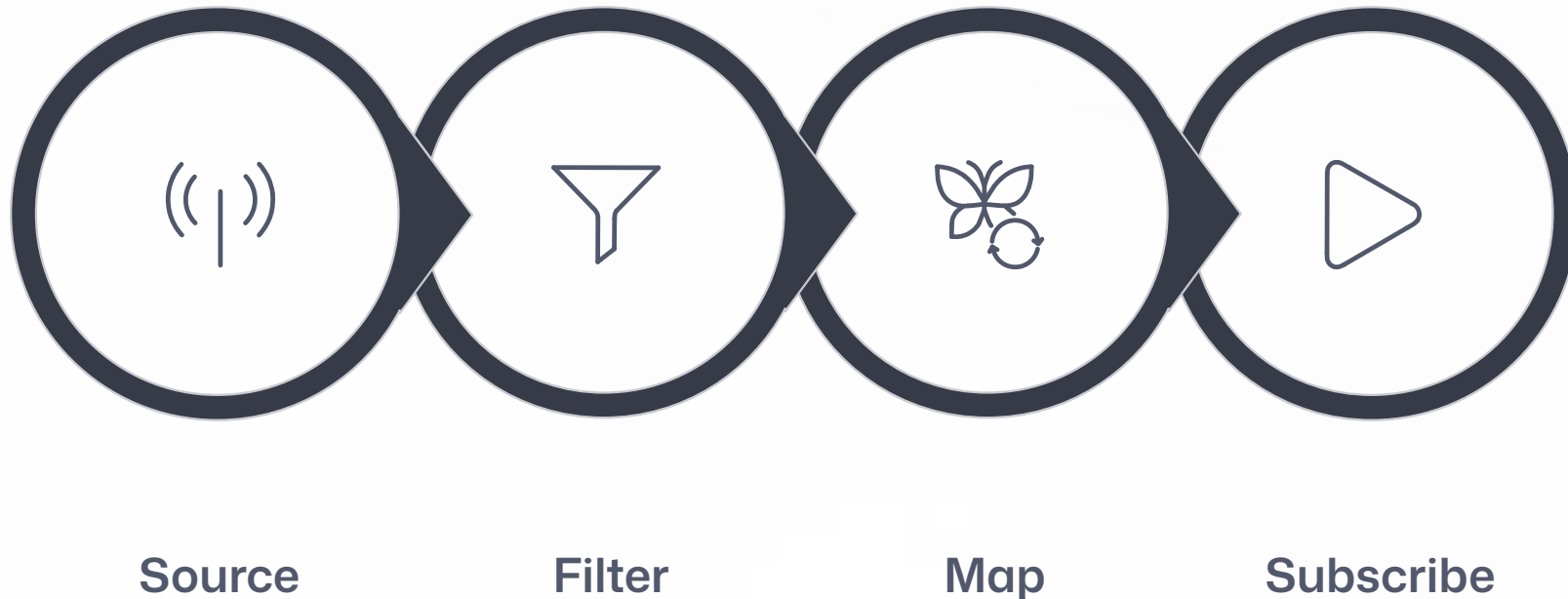
take / skip

Prend ou ignore les N premiers éléments d'un Flux.

onErrorReturn

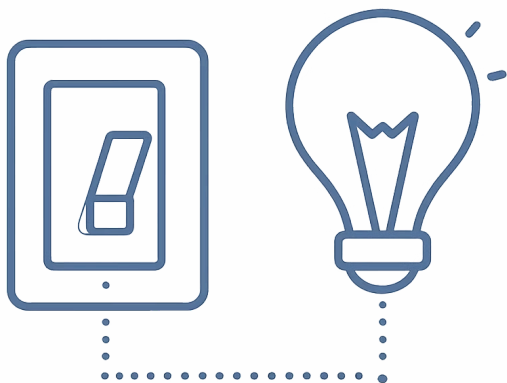
Fournit une valeur de repli en cas d'erreur dans le flux.

La chaîne d'opérateurs : un pipeline



Les opérateurs se chaînent pour former un **pipeline déclaratif**. Rien ne s'exécute tant qu'un `subscribe()` n'est pas appelé — c'est le principe de la **souscription paresseuse (lazy)**.

L'importance du Subscribe



Un Flux ou un Mono est **froid par défaut** : il ne fait rien tant que personne ne s'y abonne.

Appeler `.subscribe()`, c'est **déclencher l'exécution** du pipeline. C'est comme brancher la prise d'un appareil électrique.

- ⓘ Dans Spring WebFlux, le framework appelle `subscribe()` automatiquement lorsqu'il retourne une réponse HTTP — vous n'avez généralement pas à le faire manuellement.



Partie 5

Travaux Pratiques

TP 1 – Introduction à Reactor

Avant de toucher à WebFlux, familiarisez-vous avec Project Reactor directement.

1 Créer un Flux et un Mono

Utilisez `Flux.just()`, `Mono.just()` et `Flux.range()` pour créer vos premiers flux.

2 Appliquer des opérateurs

Chaînez `map`, `filter` et `take` pour transformer vos données.

3 S'abonner et observer

Appelez `.subscribe()` avec des lambdas pour afficher les valeurs, gérer les erreurs et détecter la complétion.

TP 2 – Premier endpoint WebFlux

Objectif

Créer une API REST simple avec Spring WebFlux qui expose une liste d'utilisateurs en mode réactif.

- Annoter le contrôleur avec `@RestController`
- Retourner un `Flux<User>` depuis un `@GetMapping`
- Tester avec `WebTestClient`

Structure du projet

UserController

Retourne `Flux<User>`

UserService

Logique métier réactive

UserRepository

Accès données non-bloquant

TP 3 – Gestion des erreurs et opérateurs avancés

Approfondissez votre maîtrise en traitant les cas d'erreur et en composant des pipelines plus complexes.

→ Simuler une erreur

Utilisez `Mono.error()` et gérez-la avec `onErrorReturn()` ou `onErrorResume()`.

→ Combiner des flux

Utilisez `Flux.zip()` pour combiner deux sources de données en parallèle.

→ Streaming SSE

Exposez un endpoint Server-Sent Events avec `MediaType.TEXT_EVENT_STREAM_VALUE` pour envoyer des données en continu.

Récapitulatif de la formation

Réactivité

Paradigme asynchrone, non-bloquant, orienté flux de données

Reactor

Mono & Flux, opérateurs, subscribe paresseux

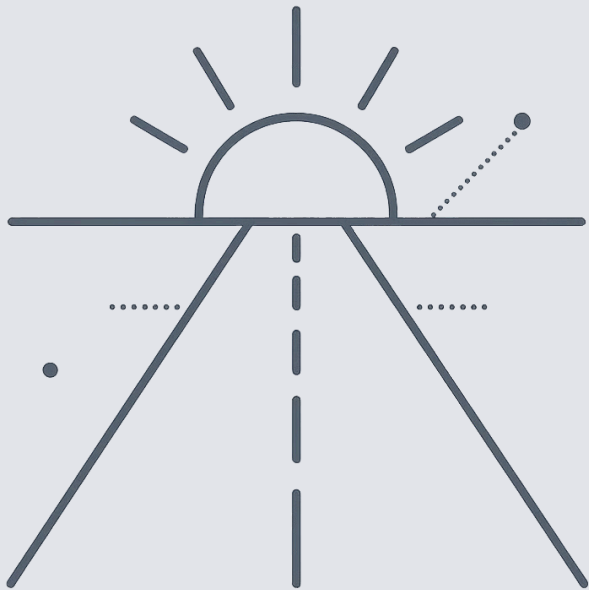
WebFlux

Event Loop, haute concurrence, streaming temps réel

Pratique

API réactive, gestion d'erreurs, SSE

- ✔ Vous avez posé les bases solides de la programmation réactive avec Spring WebFlux. La pratique régulière est la clé pour intégrer ces concepts naturellement.



Et maintenant ?

Continuez à progresser avec ces prochaines étapes :

1

R2DBC

Accès réactif aux bases de données relationnelles

2

WebClient

Client HTTP non-bloquant pour consommer des APIs

3

Spring Security Réactif

Sécuriser vos endpoints WebFlux

4

Tests réactifs

Maîtriser `StepVerifier` et `WebTestClient`